

Scaling laws for training and inference compute

How to spend a fixed budget: the Chinchilla frontier, the serving bend, and the new test-time axis

Chapter 6 ended with a number — say the 8.4×10^{22} FLOPs of the worked run — and immediately started the next argument: two engineers, same budget, two plans. One wants 13B parameters on 1.1T tokens; the other wants 7B on 2T. Five years ago this was settled by taste and courage. Today it is settled by a formula, and the formula says something precise: at *equal training compute*, the fitted loss surface puts the 13B plan ahead by 0.007 nats — essentially a tie — but the 7B model will be **half as expensive on every token it ever serves**, and if it serves more than roughly a trillion tokens in its life, the smaller plan wins on *total* compute even after buying extra training tokens to close the quality gap. That sentence is this chapter: quality as a smooth, fittable function of size and data (§7.1), bent by serving economics (§7.2), extended by a genuinely new axis — compute spent *while thinking* (§7.3) — and unified into the one budget conversation that modern model planning actually is (§7.4). Then we make it yours: how to fit these curves on your own data, at small scale, for the price of a few ablations (§7.5).

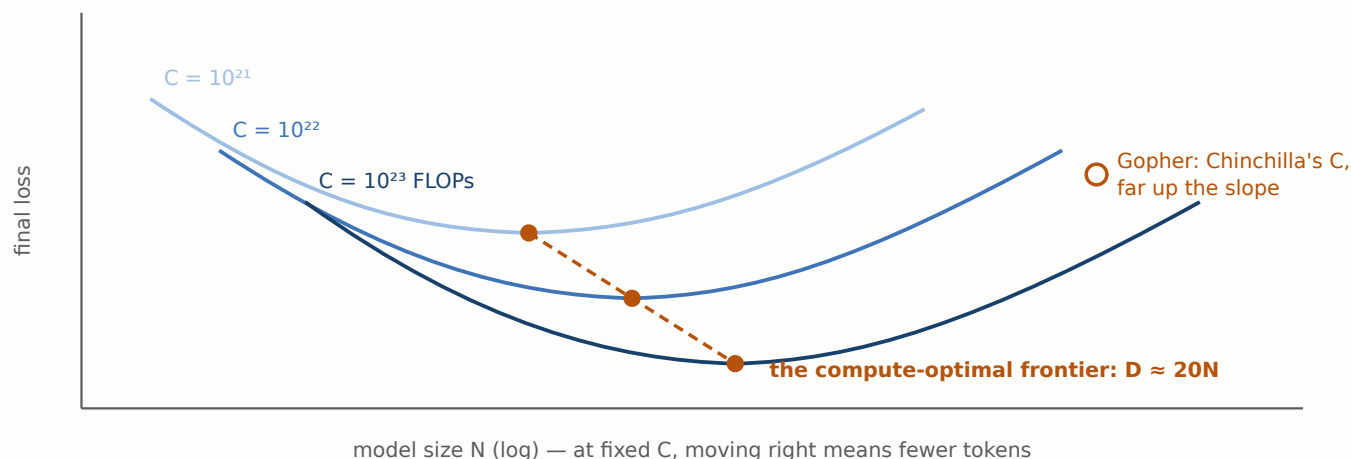
7.1 Chinchilla in one figure

The founding observation is almost embarrassing in hindsight: hold training compute fixed, vary how you split it between parameters N and tokens D , and the final loss traces a smooth valley. Kaplan and colleagues showed the smoothness in 2020¹; Hoffmann and colleagues showed in 2022 that the field was standing on the wrong side of the valley.² Their fitted surface —

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad E=1.69, \ A=406.4, \ B=410.7, \ \alpha=0.34, \ \beta=0.28$$

— reads as three terms with names: E is the **irreducible entropy** of the data itself (Chapter 2's floor — no model reaches zero loss on real text); the A term is what you pay for **too few parameters**; the B term for **too few tokens**. Minimize along a fixed-compute constraint $C = 6ND$ and the two deficits balance at the famous ratio: about **20 tokens per parameter** at the paper's budgets. The era's monuments were far off that mark — Gopher spent Chinchilla's compute on 280B parameters and 300B tokens; the same money as 70B-on-1.4T lands 0.06 nats lower on the fitted

surface, and the real Chinchilla beat the real Gopher across the benchmark suite. An entire generation of models was parameter-rich and data-starved.



Valleys are shallow near the floor — 2× off-optimal costs little; 8× off (the pre-2022 norm) cost a generation.

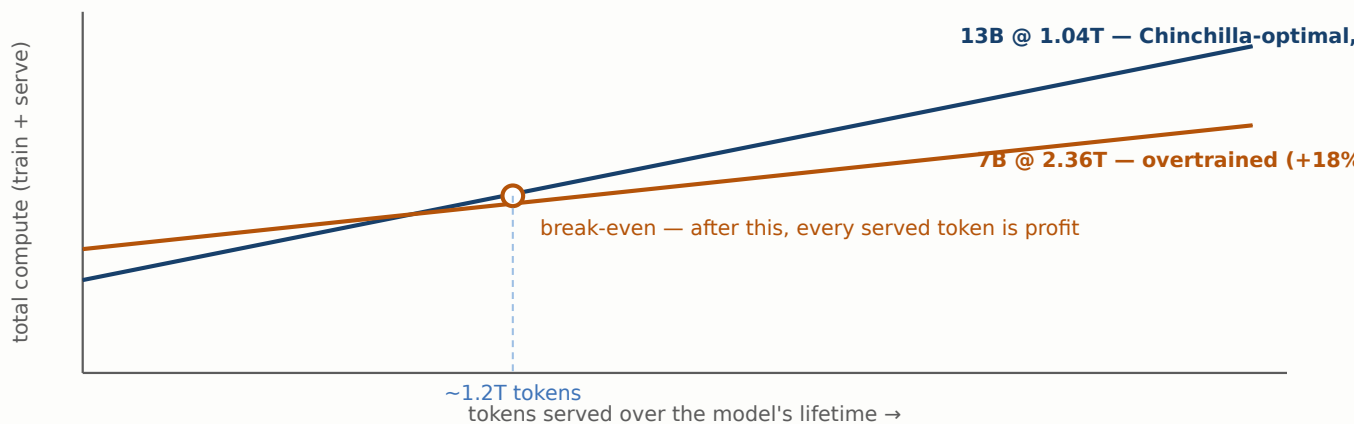
Figure 7.1 — IsoFLOP valleys and the frontier through their floors. At each budget, loss versus size is a valley; the floors line up along $D \approx 20N$. Two honest footnotes the caption should carry: the exact exponents are fit-dependent (the paper's own three estimation approaches disagree at large C), and the valleys flatten near the bottom — which is precisely what §7.2 exploits.

7.2 The overtraining regime: why inference bends the curve

Chinchilla optimizes the wrong objective for anyone who ships. It minimizes loss for a fixed *training* bill — but Chapter 5 already told you a model is trained once and serves forever, at $2N$ FLOPs per generated token. Write the true objective and the optimum moves:

$$C_{\text{total}} = \underbrace{6ND_{\text{train}}}_{\text{paid once}} + \underbrace{2ND_{\text{serve}}}_{\text{paid forever}}$$

Every parameter you carry taxes both terms. So for a target *quality*, the shipping question becomes: can a smaller N reach the same loss with more tokens, and does the serving discount repay the training surcharge? The fitted surface answers with numbers. At our 8.4×10^{22} budget the surface's floor sits near 13B; a 7B can match that floor's loss by training on 2.36T tokens instead — an 18% training surcharge — after which it serves at roughly half the FLOPs per token. Break-even arrives after about **1.2T served tokens**; every token after that is profit, forever. That arithmetic — formalized as inference-aware scaling laws³ — is why the era's workhorses sit absurdly far right of the Chinchilla ratio (Chapter 5's 8B at 94× past it), why the valleys' flatness is a gift rather than a rounding error, and why the answer to §7.0's meeting is: *the 7B, unless this model will barely be used — in which case why are you training it?*



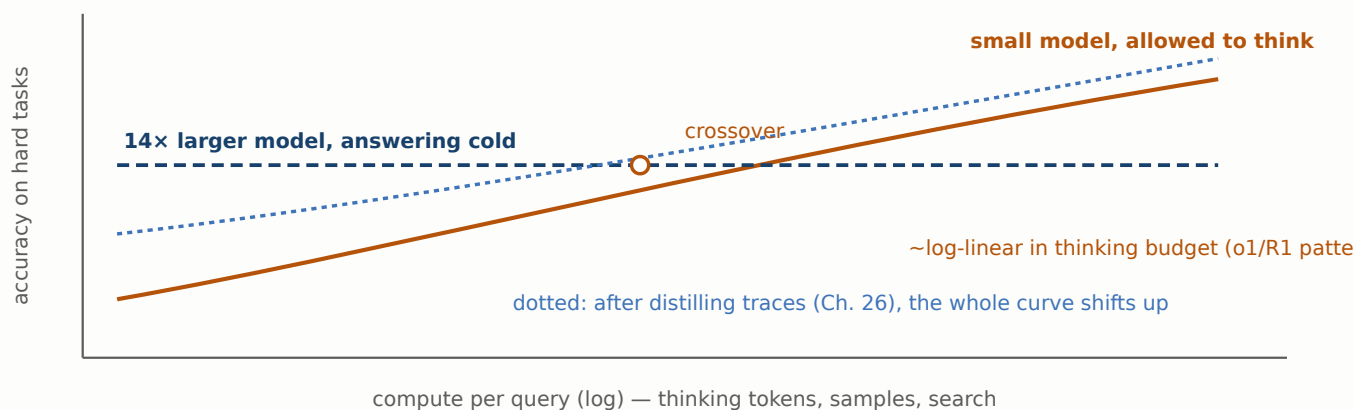
Computed from the §7.1 fit at $C = 8.4 \times 10^{22}$ and 2N FLOPs per served token. A popular assistant serves 1.2T tokens in week

Figure 7.2 — The serving bend, with the chapter's own numbers. Equal quality, two intercept-slope trades: the compute-optimal model wins the training bill, the overtrained small model wins the lifetime. The only input you need from the business side is the one axis label: how much will this model be used?

7.3 Test-time compute: the third axis

Until recently the budget had two dials. Then the reasoning models added a third, and it changes the geometry of the whole planning problem: you can buy accuracy *per query, at inference*, by letting the model compute longer before answering. The mechanisms are ones you have already met or will: **longer chains of thought** — the o1/R1 result that accuracy on hard math climbs steadily, roughly log-linearly, as the model is allowed more thinking tokens⁴⁵; **parallel sampling** — best-of- n with a verifier or reward model picking the winner (Chapter 27's rejection sampling, promoted to inference time); and **search** — tree-structured exploration scored by process reward models. The striking empirical claim, tested carefully at smaller scales: for many problems, compute spent thinking beats the same compute spent on parameters — a small model with an optimally-allocated thinking budget can outperform a model $\sim 14\times$ larger that answers cold.⁶

But price the axis honestly, because its economics are unlike training's. Test-time compute is paid *per query, forever*, and it multiplies serving cost brutally: a reasoning model that emits 10,000 thinking tokens to answer what a tuned model answers in 500 has made each query $20\times$ more expensive — §7.2's arithmetic with the sign flipped. The axis is a gift precisely where queries are rare and valuable (hard math, agentic coding, research assistance) and a trap where they are cheap and plentiful (chat at scale). Which sets up the move that closes the loop, and that you already know from Chapter 26: **let the big model think expensively once, then distill the results into weights** — R1's trajectory, where test-time discoveries became 800K traces became small models that answer well *without* the thinking bill. Test-time compute explores; distillation amortizes.



Schematic of the published patterns — the axis is real money: 10K thinking tokens per answer is a 20x serving multiplier

Figure 7.3 — The third axis, and the loop that tames it. Thinking climbs where parameters plateau — and costs per query what parameters cost once. The distillation loop (think expensively → keep verified traces → train them in) converts the amber curve's discoveries into the blue curve's cold-answer competence.

7.4 One budget: putting the three dials together

Here is the modern planning conversation, compressed. You control three spends: **pre-training compute** (permanent capability, paid once — §7.1's frontier says how to split it), **test-time compute** (per-query capability, paid always — §7.3's curves say what it buys), and **the conversion cycle between them** (distillation and trace-training, Chapter 26 — turning per-query spend into permanent weights at the cost of a data pipeline). The equilibrium depends on one product fact and one portfolio fact. The product fact is query economics: high-volume/low-value traffic pushes spend toward pre-training and overtraining (§7.2), low-volume/high-value traffic tolerates test-time spend. The portfolio fact is that the cycle compounds: each generation, the expensive thinker's outputs raise the cheap model's floor, which raises what the next round of thinking starts from — the pattern every 2025-era reasoning family (o-series → distills, R1 → R1-Distill, Qwen's flagship → its smalls) is running. Plan all three dials in one meeting, or you will optimize one leg of a triangle someone else is walking around.

7.5 Fitting your own scaling law from small runs

The published constants are fits *to their data, their tokenizer, their recipe* — and yours differ (a cleaned bilingual corpus, a different vocabulary, packing fixed). The exponents travel reasonably well; the constants do not. The good news: the protocol that produced Figure 7.1 is affordable at ablation scale, and it converts sizing debates into arithmetic *about your own stack*. Train a small grid — five model sizes at four modest budgets, 20 runs of minutes-to-hours each — and fit the surface:

```
# fit  $L(N, D) = E + A/N^a + B/D^b$  to your own ablation grid
import numpy as np
from scipy.optimize import curve_fit

# each run: (params, tokens, final held-out loss) — same data mix, tokenizer,
```

```
# schedule discipline (cosine run to completion per budget: the Kaplan/
# Chinchilla discrepancy was largely un-decayed schedules – don't repeat it)
runs = load_grid()          # ~5 sizes x 4 budgets, e.g. 50M-800M params
N, D, L = runs.N, runs.D, runs.loss

law = lambda X, E, A, B, a, b: E + A / X[0]**a + B / X[1]**b
(E, A, B, a, b), _ = curve_fit(law, (N, D), L,
                              p0=[1.5, 100, 100, 0.3, 0.3], maxfev=50_000)

# now ask it your real questions:
L_at = lambda N_, D_: E + A / N_**a + B / D_**b
print(L_at(7e9, 2.0e12), L_at(13e9, 1.08e12)) # the meeting, settled with YOUR
curve
# On a synthetic check, this grid recovers the generating exponents to ~2%
# and extrapolates 7B@2T to within ~0.01 nats – and that residual is the
# honest error bar to carry into the meeting (see pitfalls).
```

Protocol discipline, learned the hard way by everyone who has done this: keep *everything except N and D* fixed across the grid (mix, tokenizer, schedule-to-completion, batch scaling rule); run each budget's cosine to its end — truncated schedules gave Kaplan's fit its famously too-parameter-hungry slope²; and extrapolate no more than ~2 orders of magnitude in compute, carrying the fit residual as an error bar. Two orders is exactly small-ablations-to-7B — which is why this section exists in a book for teams that cannot afford to find out at full scale.

7.6 What the big labs do

Table 7.1 — Scaling-law practice at the frontier.

LAB / MODEL	PRACTICE	THE LESSON
Chinchilla ²	400+ runs, three estimation approaches, D≈20N headline	The correction that defined the field — and its approaches already disagreed at large C: optima are fits, not physics
Llama 3 ⁷	own isoFLOP fits predicted 402B optimal at 3.8×10 ²⁵ ; trained 405B — then a downstream- task law linking loss to benchmark accuracy	Labs re-fit on their own data, and increasingly fit loss → capability, not just compute → loss
DeepSeek ⁸	scaling laws for batch size and learning rate as functions of C, before the model laws	Hyperparameters scale too — Chapter 17 uses exactly this
Qwen / small- model era	7B-class models at 90×+ past the ratio (Ch. 5)	§7.2 is industry consensus: serving bends everything

LAB / MODEL	PRACTICE	THE LESSON
MiniCPM ⁹	WSD schedules to reuse one run for many budget points on the curve	Fitting your own law is getting cheaper — no excuse left
o1 / R1 ⁴⁵	published test-time scaling curves; distill the results (Ch. 26)	The third axis is real, log-linear, and feeds the conversion cycle

7.7 Practical considerations

Loss is the fit's currency; capability is the product's. The surface predicts held-out loss superbly and benchmarks only via a second, noisier mapping — small loss gaps can hide meaningful capability gaps and vice versa, especially across data mixes. Llama 3's two-stage law (compute → loss → accuracy) is the mature pattern⁷; at minimum, validate your fit's ranking on a couple of real checkpoints before betting the run on it.

Data constants are diet constants. Change the mix (more code, more Spanish, Chapter 27's synthetic tokens) and E, A, B move — better data lowers the whole surface, which is Chapter 10's "cheapest lever" said in this chapter's language. Refit when the diet changes materially; and when tokens are scarce, remember repetition has its own law: up to ~4 epochs of good data behaves nearly like fresh data, decaying after¹⁰ — often the difference between "we lack tokens" and "we lack unique tokens."

Common pitfalls

Treating 20:1 as physics.

It is one fit's optimum at one era's budgets, on one diet. The durable content is the *method* and the valley's shape, not the integer.

Extrapolating heroically.

Two orders of compute, with error bars, or you are decorating a guess. The fit's residual at held-out grid points is your uncertainty — carry it.

Comparing at training cost when you mean total cost.

Every sizing argument that omits $2ND_{\text{serve}}$ — and, for reasoning products, the thinking multiplier — reaches the wrong optimum on purpose.

Grid runs with drifting recipes.

One changed schedule or mix inside the grid and the fit learns your inconsistency, not the law.

Ignoring seed noise at small scale.

50M-parameter runs are noisy; repeat the smallest points or the exponents inherit the noise.

Budgeting test-time compute nowhere.

Reasoning modes multiply serving cost 10–50×; a plan that prices tokens without pricing *thinking* tokens is off by that factor.

7.8 Decision framework: sizing a model for a fixed budget

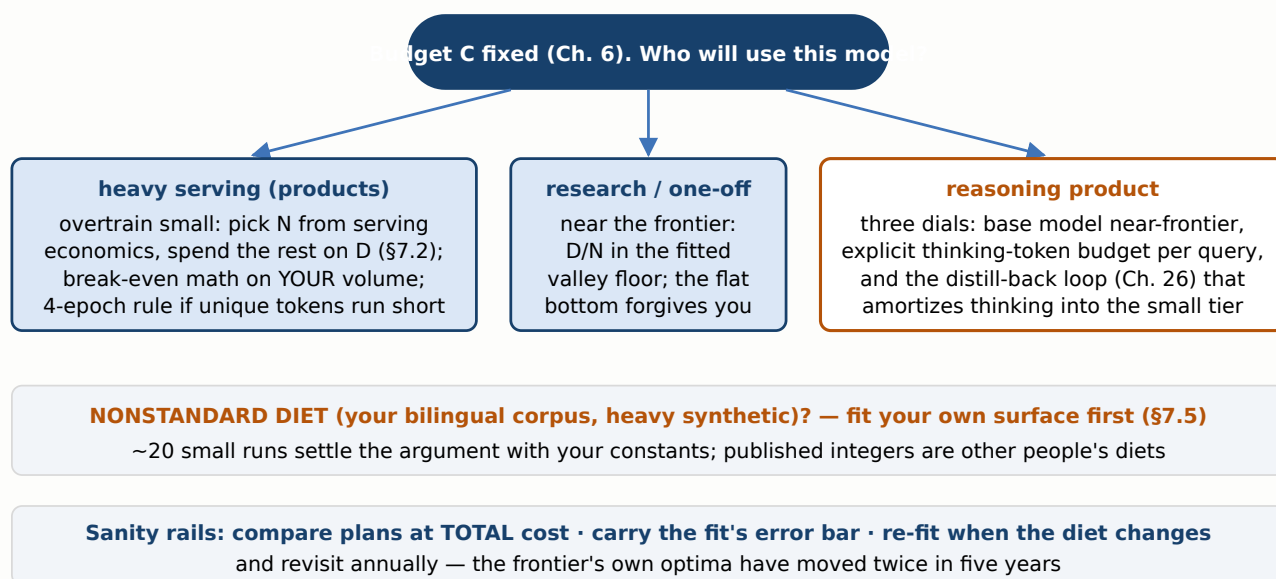


Figure 7.4 — Sizing under a fixed budget. Serving volume picks the regime, reasoning products add the third dial and its amortization loop, and a nonstandard diet — this book's audience, usually — earns twenty small runs before anyone quotes someone else's constants.

Cost-benefit, bluntly. Knowing the published laws: free, and sufficient for standard diets at standard scales. Fitting your own: ~20 small runs — real but modest money (\$6.2's ablation line already contains it) — and it retires the single most expensive class of mistake available to a training team, the mis-sized flagship. The overtraining move: +10–25% training for –40–50% serving forever wherever volume is real. Test-time budgets: the cheapest capability rental there is for rare hard queries, and the most expensive default imaginable for chat. **How this scales:** the laws are the rare tool that gets *more* reliable as you grow (smoother curves, more leverage per decision), while the penalty for skipping them grows with the same exponent as your budget — at 10^{25} FLOPs nobody sizes by taste, and the discipline of this chapter is how you behave like that lab at 10^{22} too.

7.9 Summary

Quality is a smooth, fittable function of parameters and data: $L = E + A/N^\alpha + B/D^\beta$, valleys at fixed compute, floors near $D \approx 20N$ in the founding fit — and the Gopher-vs-Chinchilla episode as permanent proof that being far off the frontier wastes a fortune. But the frontier optimizes the training bill alone, and shipping adds $2N$ per served token

forever: the serving bend makes overtrained small models the workhorse regime (this chapter's own numbers: a 7B matches the 13B floor for +18% training and breaks even after $\sim 1.2T$ served tokens). Test-time compute is the genuinely new third axis — thinking tokens buy accuracy roughly log-linearly, small thinkers cross large cold models, and the per-query price is savage — tamed by the distillation loop that converts expensive thinking into cheap weights, the cycle every frontier reasoning family now runs. Plan all three dials in one budget; fit your own surface when your diet isn't theirs (twenty small runs, schedules run to completion, two orders of extrapolation, error bars carried); compare plans at total cost including thinking tokens; and hold the era's meta-lesson: the optima have moved twice in five years, so the durable asset is not any ratio — it is the habit of measuring your own curve before spending your own millions.

References

1. Kaplan, J., et al. (2020). Scaling Laws for Neural Language Models. arXiv:2001.08361.
2. Hoffmann, J., et al. (2022). Training Compute-Optimal Large Language Models. *NeurIPS*. arXiv:2203.15556. The fitted surface used throughout ($E=1.69$, $A=406.4$, $B=410.7$, $\alpha=0.34$, $\beta=0.28$), the $D \approx 20N$ frontier, and the schedule explanation of the Kaplan discrepancy.
3. Sardana, N., Portes, J., Doubov, S., & Frankle, J. (2024). Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws. *ICML*. arXiv:2401.00448.
4. OpenAI (2024). Learning to Reason with LLMs (o1). openai.com. Published train-time and test-time compute scaling curves for reasoning.
5. DeepSeek-AI (2025). DeepSeek-R1. arXiv:2501.12948. Thinking-length growth under RL; the distillation loop of §7.3–7.4.
6. Snell, C., Lee, J., Xu, K., & Kumar, A. (2024). Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters. arXiv:2408.03314. Compute-optimal test-time allocation; small-model-with-search vs $\sim 14\times$ larger baselines.
7. Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783. Own isoFLOP fits (402B optimal at 3.8×10^{25}) and the two-stage loss $\rightarrow$ benchmark law.
8. DeepSeek-AI (2024). DeepSeek LLM: Scaling Open-Source Language Models with Longtermism. arXiv:2401.02954. Batch-size and learning-rate scaling laws.
9. Hu, S., et al. (2024). MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies. arXiv:2404.06395. WSD schedules for cheap multi-budget law fitting.
10. Muennighoff, N., et al. (2023). Scaling Data-Constrained Language Models. *NeurIPS*. arXiv:2305.16264. Repetition behaves near-fresh to ~ 4 epochs, decaying beyond.

Next: *Chapter 8 — The transformer as a compute object* — where the MFU that §7's conversions assumed gets measured and earned. · [Training Efficient LLMs](#)